University of Piraeus
School of Information and Communication Technologies,
Department of Digital Systems

Level: Postgraduate Programme

Network Security

Burp Suite

Supervising Professor: Prof. Christos Xenakis

Ioannis Kalaitzidis ioannis_kalaitzidis@ssl-unipi.gr

Piraeus
26/12/2025

Abstract

This thesis examines the offensive approach (Red Teaming) to web application security. Using the PortSwigger-Web Security Academy platform and the Burp Suite tool, an analysis and exploitation of vulnerabilities were carried out in a controlled environment. Specifically, SQL injection attacks were successfully carried out to retrieve hidden data, and cross-site scripting attacks were successfully executed to run arbitrary JavaScript code, demonstrating the importance of data input validation.

Contents

Introduction

This voluntary project differs significantly from the previous one (Case 7), as it marks the transition from a defensive (Blue Team) to an offensive (Red Team) approach to information systems security. Whilst in the previous scenario the primary objective was to shield the server using firewall and WAF mechanisms to repel and block malicious traffic, in the current study we are called upon to adopt the mindset and techniques of the attacker. Specifically, the focus shifts from protection to actively compromising the application, with the aim of identifying and exploiting vulnerabilities. To carry out the scenarios in practice and document SQL injection and cross-site scripting attacks using the Burp Suite tool, the specialised training environment of the PortSwigger-Web Security Academy will be utilised. This choice was deemed optimal as it provides direct access to realistic scenarios involving vulnerable applications, allowing the focus to be placed on analysing the attacks without the complexity and limitations that would be imposed by the installation and configuration of a local vulnerable server. Finally, it should be noted that the selected labs fall into the ‘Apprentice’ category, fully meeting the requirement set out in the brief for the documentation of vulnerabilities of ‘Low’ difficulty.

Practical PART of the assignment

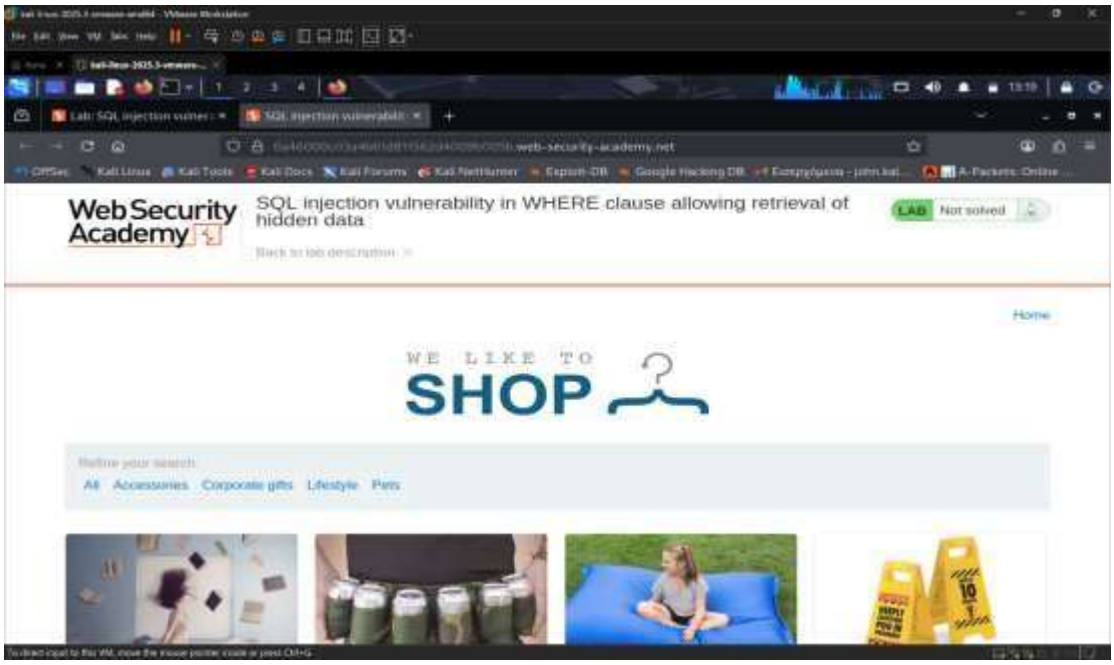
SQL injection

To carry out the assignment, we selected the PortSwigger Web Security Academy training platform, which provides a secure and controlled environment for carrying out web attacks. Specifically, we navigated to the **SQL Injection** section and selected the lab entitled:

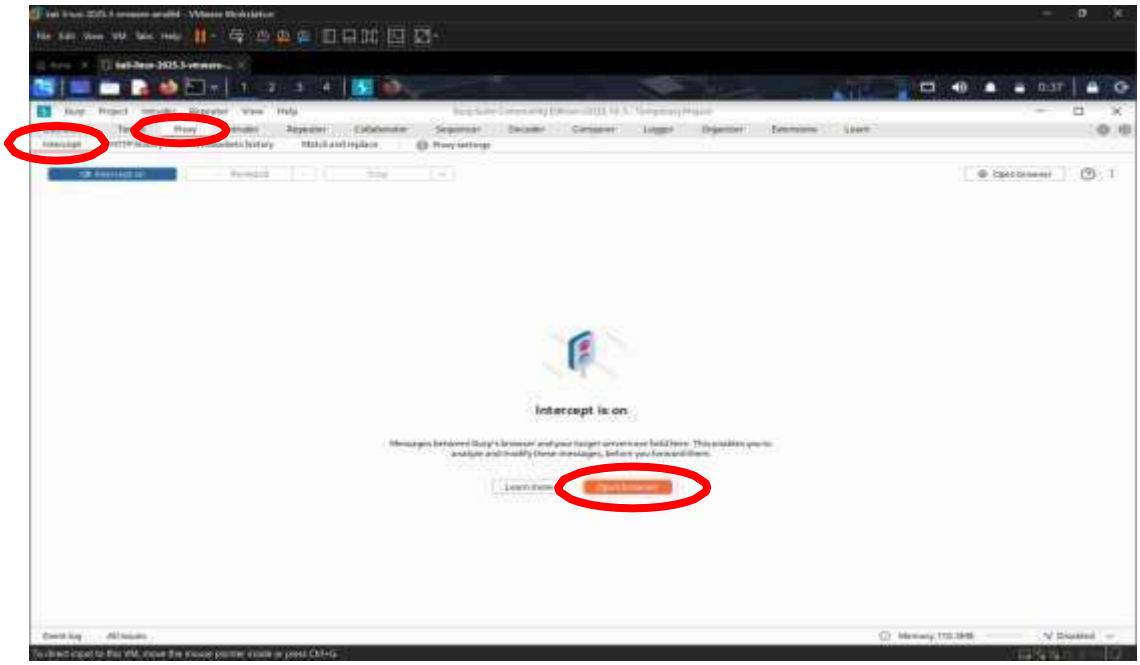
'**SQL injection vulnerability in WHERE clause allowing retrieval of hidden** data' (https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data).

T

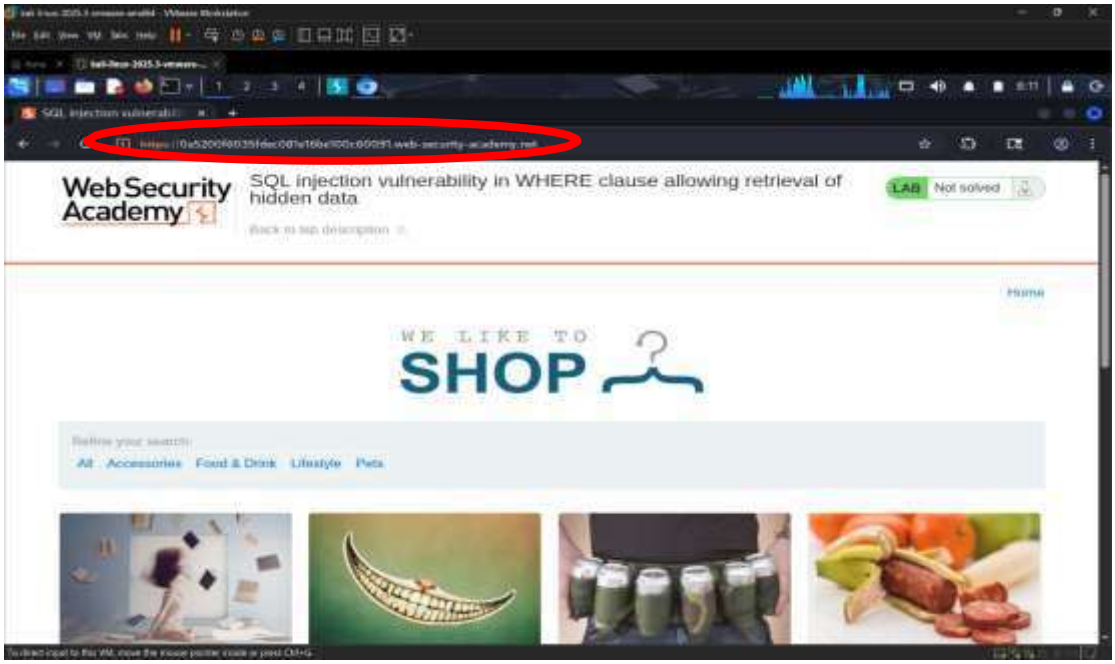
his particular workshop was deemed the best choice for the first part of the exercise, as it precisely simulates the scenario we examined from a defensive perspective in the previous assignment (Case 7). Whilst in the previous scenario our objective was to detect and block the 'OR 1=1' command via a WAF, in this scenario we are acting as attackers (Red Team). Our aim is to successfully execute the same logic (1=1) in order to manipulate the query to the database. In this way, we aim to bypass the application's controls and force the display of hidden data that would not normally be accessible to the end user.



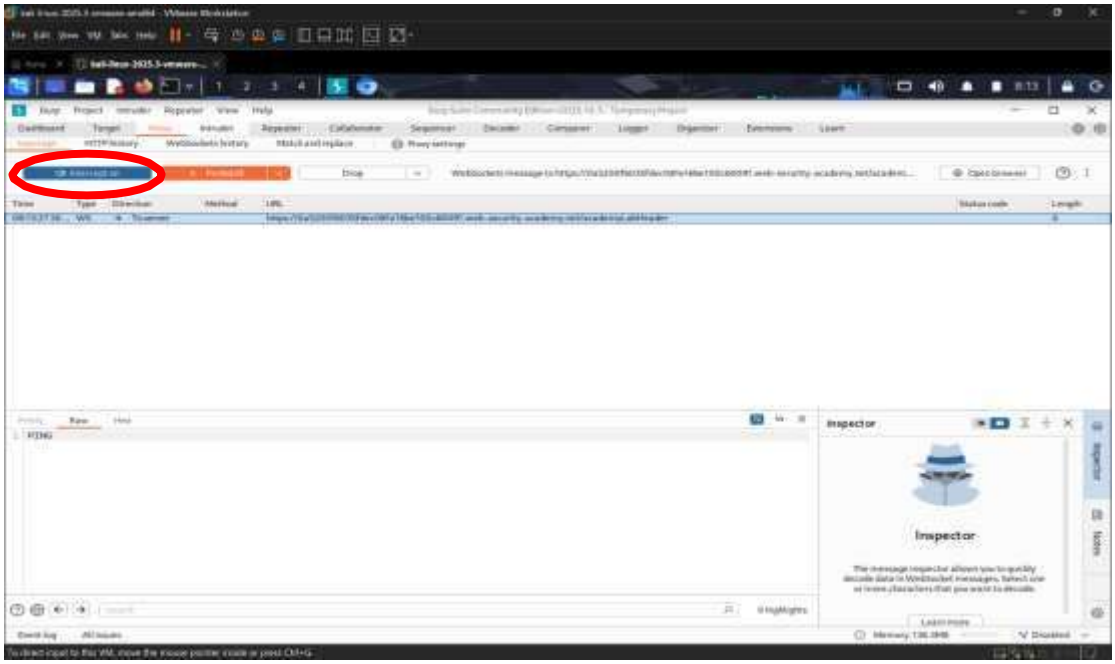
So, we open Kali Linux and search for the Burp Suite application to launch it.



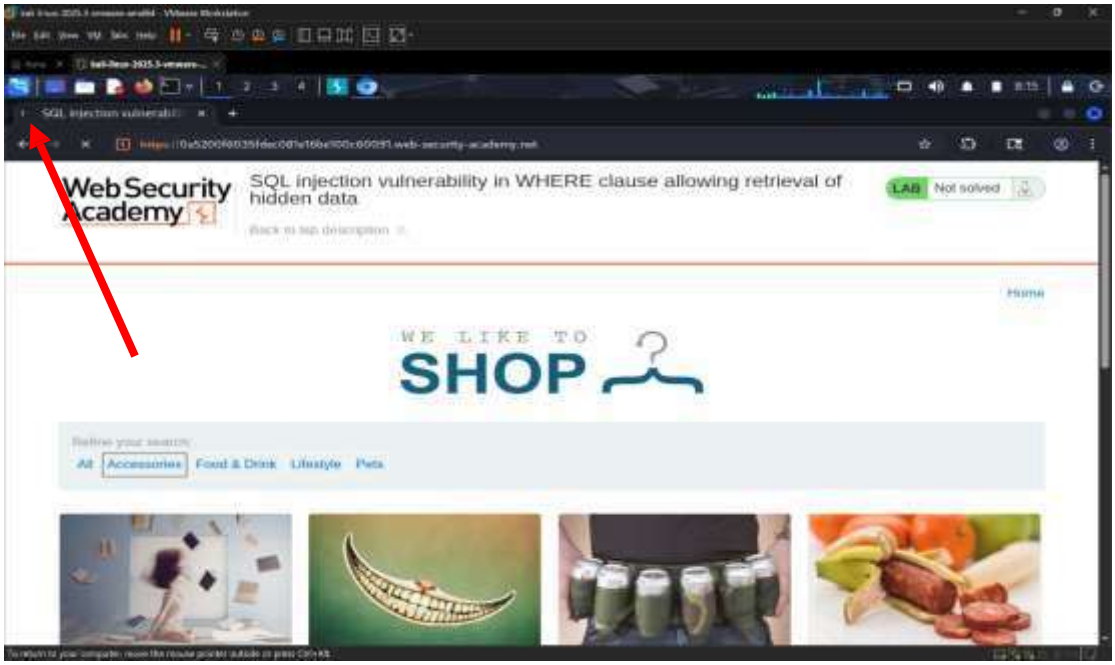
Once the application has opened, we'll select the **'Proxy'** tab and then the **'Intercept'** tab. We'll click the **'Open Browser'** button, which will open the application's dedicated browser, and paste in the URL from this particular lab. We used Burp's built-in browser because it is ready to use. This way, we avoided complex network configurations, as the traffic automatically passes through the tool.



We will now return to the Burp Suite application and enable the **'Intercept on'** feature as shown below.

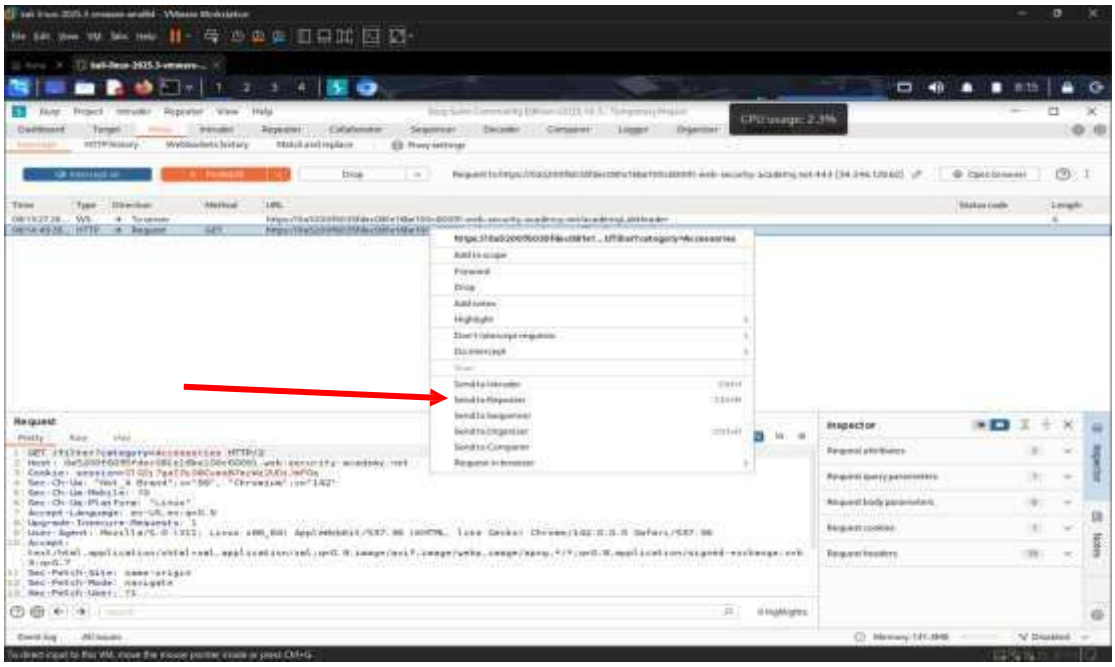


Once we've do this, we'll return to the browser of Burp:

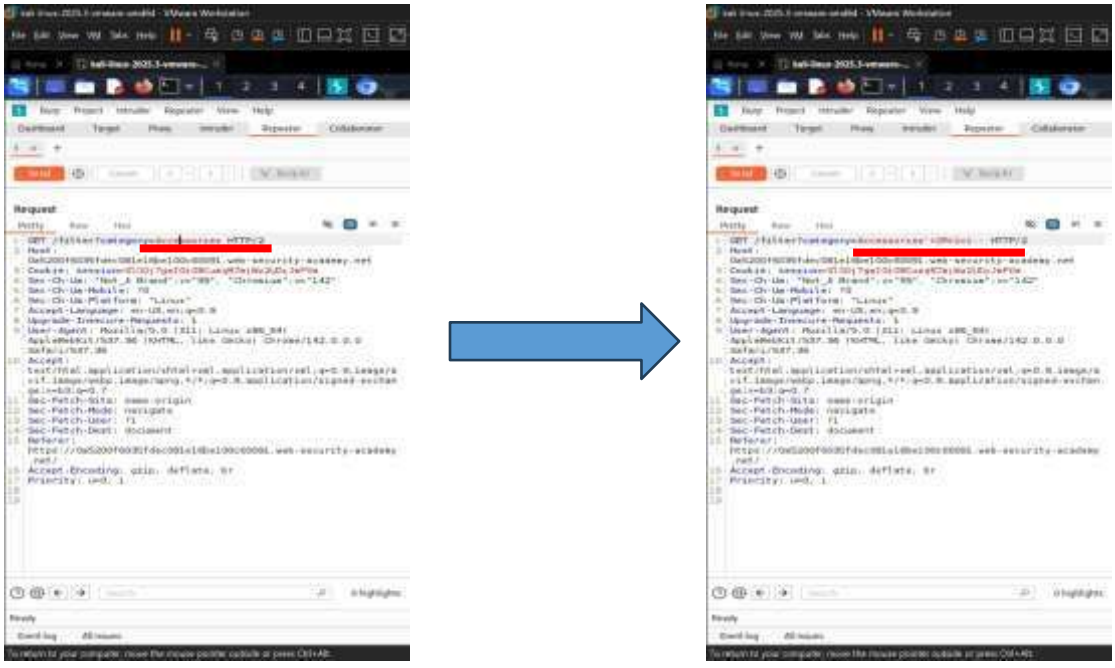


During the reconnaissance phase, we navigate to the lab's home page and interact with the application by selecting a product category (specifically the 'Accessories' category). This action triggers the sending of an HTTP request to the server. Our aim is to identify and intercept this specific request using Burp Suite, so that

to check whether the parameter is is vulnerable to SQL injection.



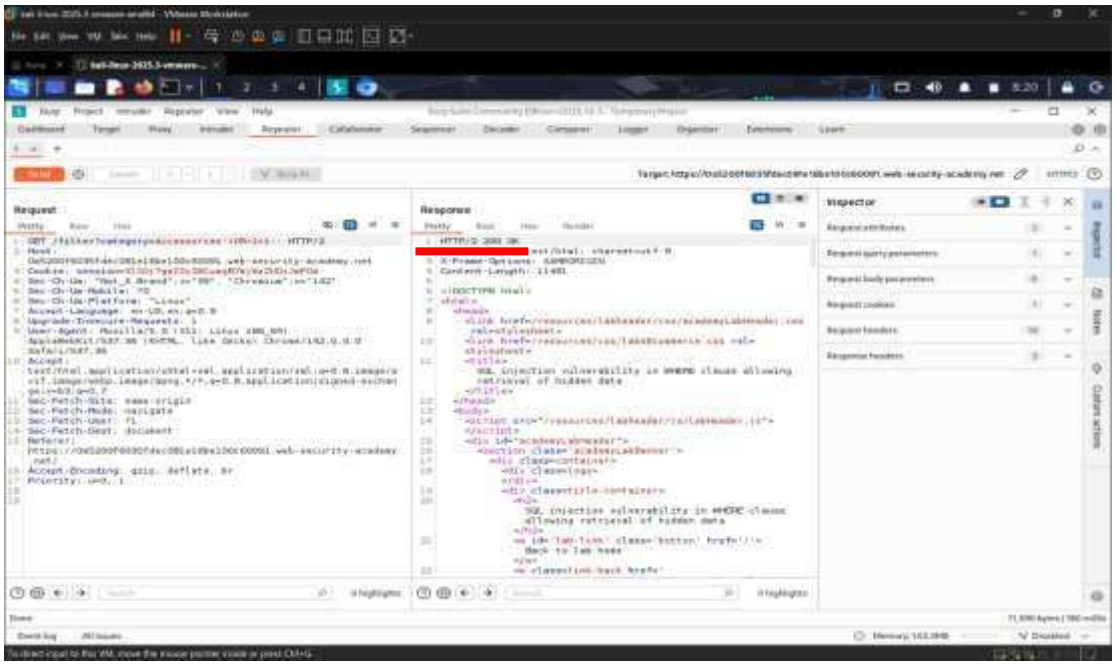
Once we enable the Intercept function, the HTTP request is successfully intercepted before it reaches the server. We confirm that this is the request that calls up the page displaying the results for the 'Accessories' category. We then right-click on the request and select **'Send to Repeater'**. This transfers the request to a controlled testing environment, where we can modify the parameters and resend the request multiple times to test the system's response to the SQL injection attack.



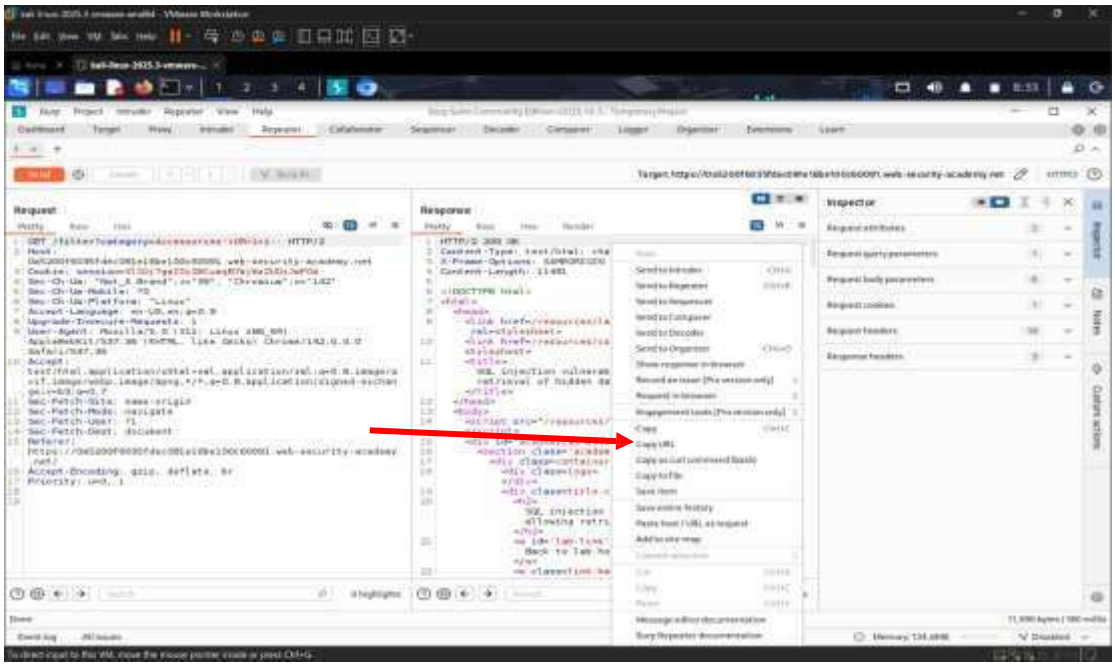
We identify the parameter category in the line of the request. We modify its value, replacing ‘Accessories’ with the attack payload: ‘OR 1=1 --’.

Payload analysis:

- The single quote (') is used to close the alphanumeric string expected by the database.
- The condition ‘OR 1=1’ is a logical statement that is always true.
- The dashes (--) indicate a comment in SQL, nullifying any part of the query that follows, in order to avoid syntax errors.



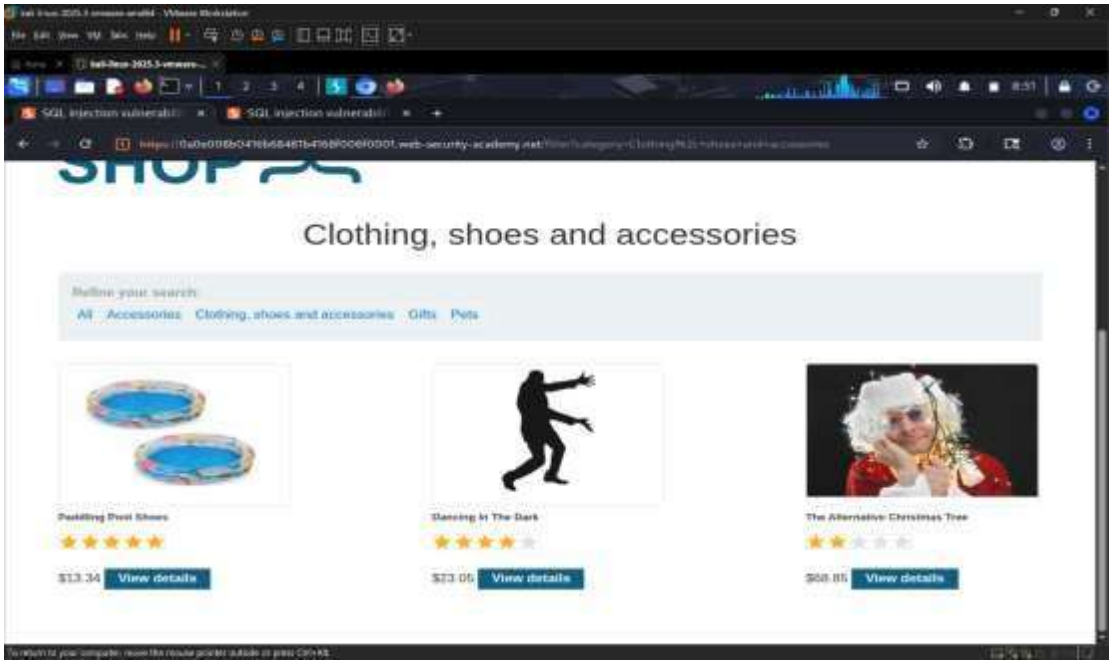
As shown in the underlined section of the image (Response), the server responded with a **200 OK** status code. This indicates that the SQL injection attack was executed correctly and did not cause a syntax error in the database. We also note that the size of the response has increased, as the webpage now returns data for all products in the database and not just for that specific category.



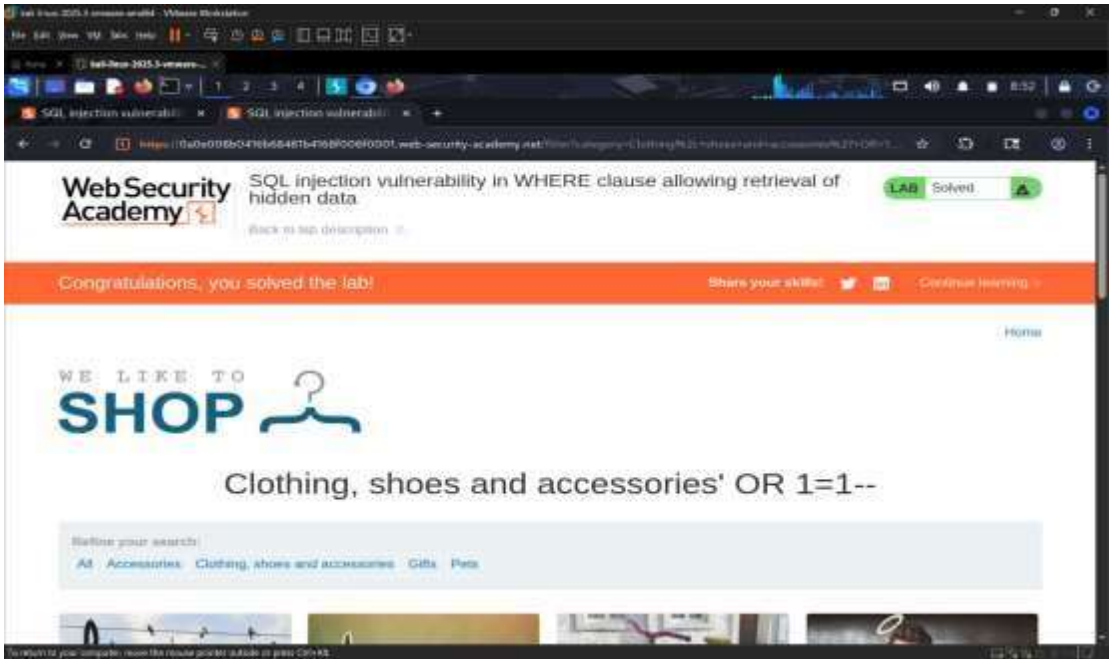
Having confirmed the success of the attack via the response codes in the repeater, the next step was visual verification in the browser. To achieve this, we disabled the **Intercept** function in the Proxy tab of Burp Suite (which we had enabled earlier), so as to allow data to flow freely. We then entered the modified URL (Copy URL) directly into the browser's address bar.

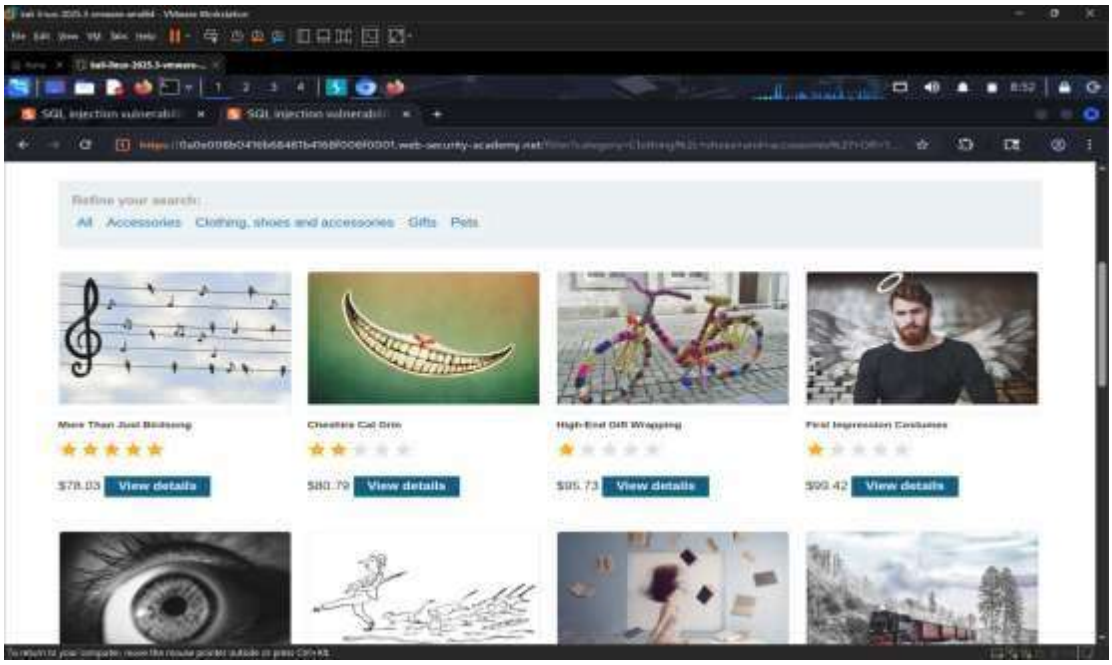
For final verification, we compared the application's behaviour before and after the attack. In the initial search (image below), the application filters the results correctly, displaying only products belonging to the

category ‘Clothing, shoes and accessories’.



After entering the payload OR 1=1, we observe that the results now display all products in the database, regardless of category, as well as products that were likely hidden or unpublished.





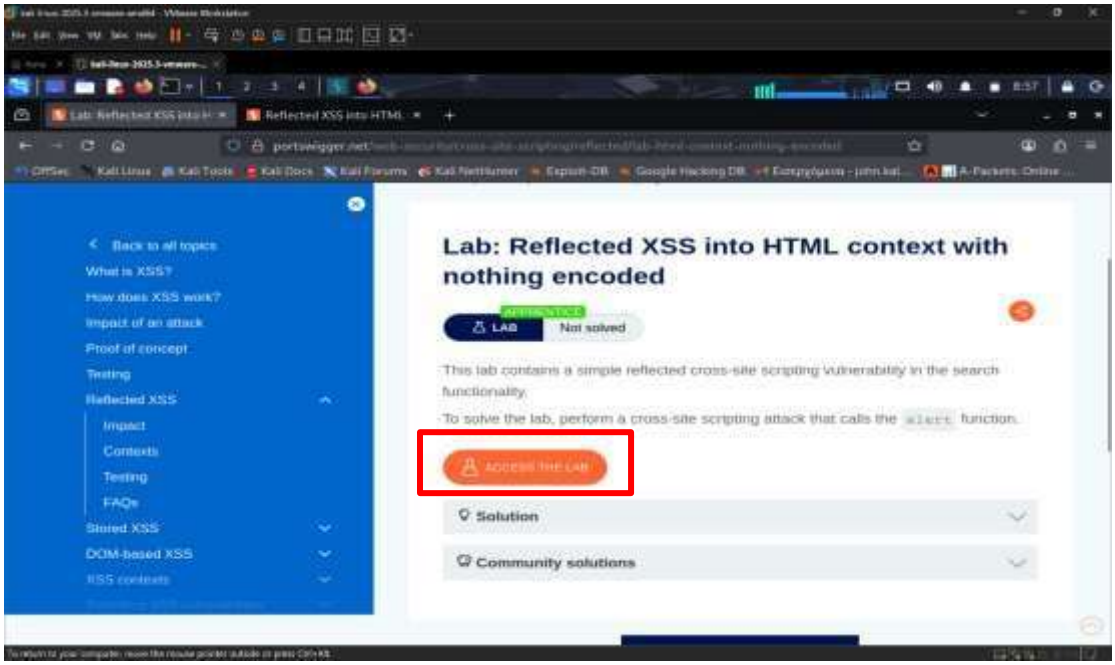
Conclusion on the SQL Injection Section

Through this specific attack, it was demonstrated that the application does not perform adequate validation on the data entered by the user. By entering the condition 'OR 1=1--', we managed to alter the logic of the SQL query sent to the database. Because the condition 1=1 is always evaluated as true, the database stopped checking the product category and returned all records from the table. This constitutes a serious breach of confidentiality, as it allows unauthorised users to access data that the administrator had designated as hidden or restricted.

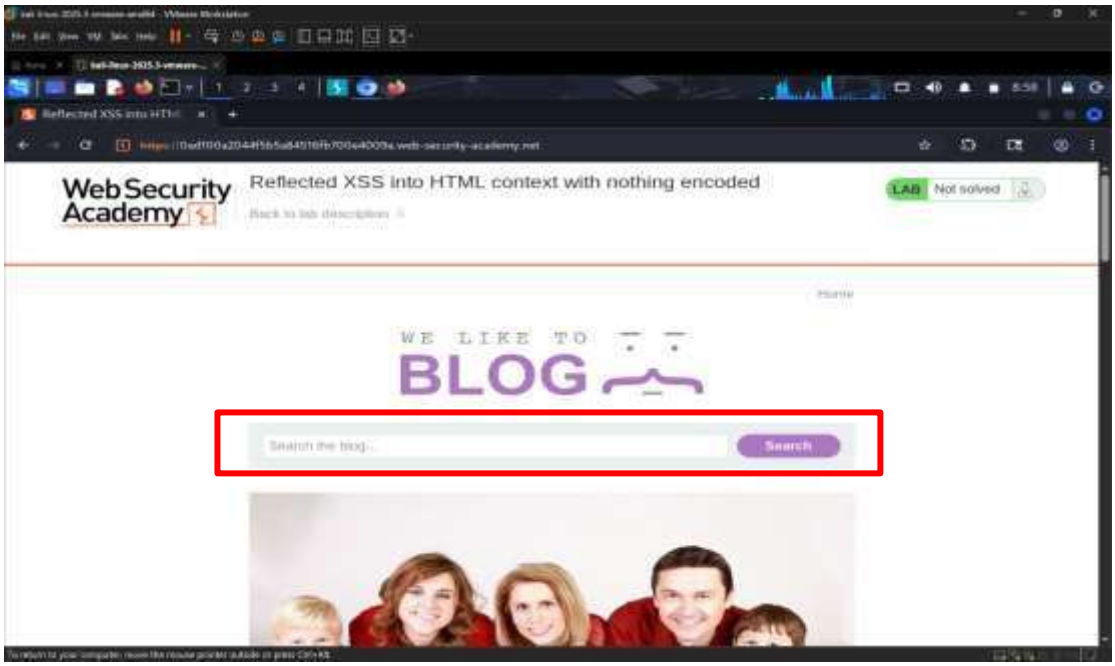
Cross-site scripting attacks

For the second part of the assignment, we moved on to the Cross-Site Scripting (XSS) attacks category. Specifically, we chose the workshop entitled 'Reflected XSS into HTML context with nothing encoded' (https://portswigger.net/web-security/cross-site-scripting/reflected/lab-html-context-nothing-encoded). The aim of this exercise is to identify an input vector where the application 'mirrors' the

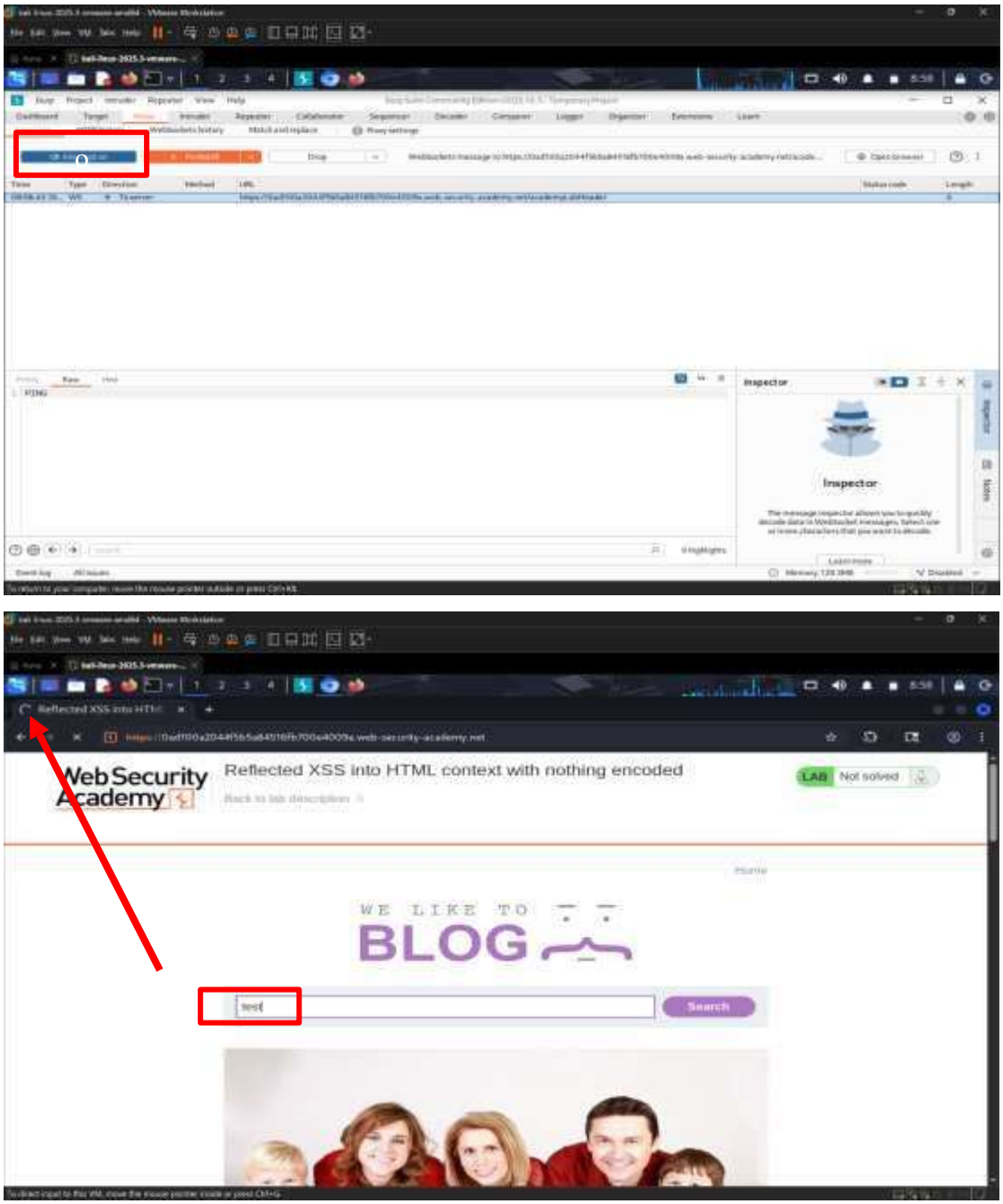
back to the browser without proper filtering. This will allow us to inject and execute arbitrary JavaScript code.



By clicking 'Access the Lab', we were taken to the environment of the vulnerable application, which simulates a blog. During the reconnaissance phase, we identified the search bar.



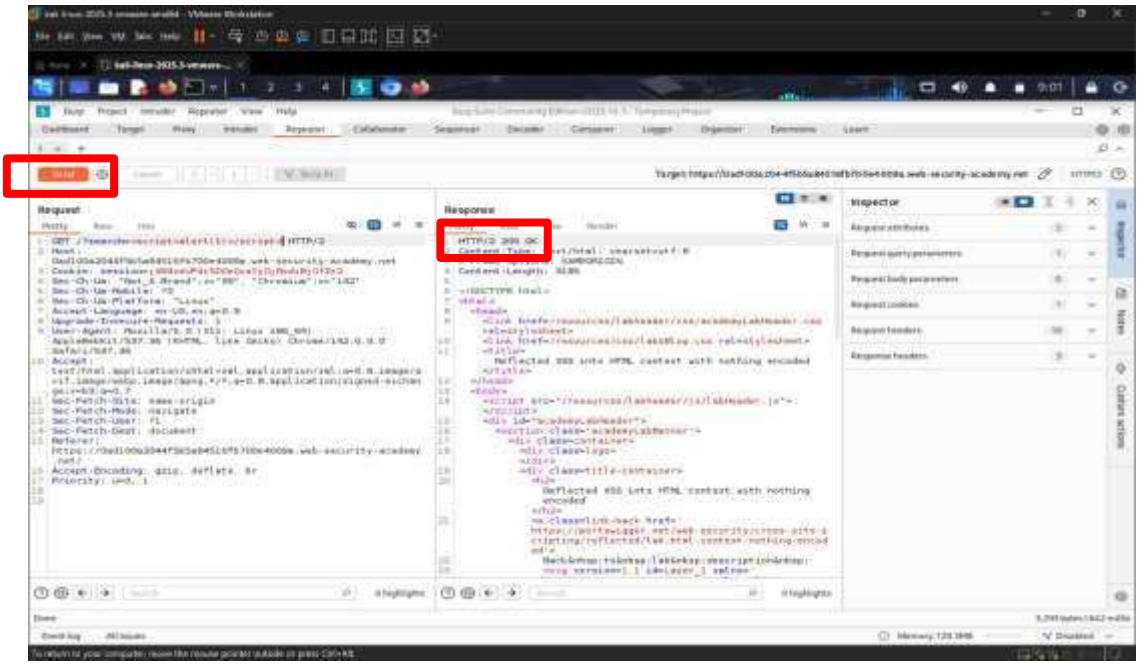
The search bar is the primary point of interest for a reflected XSS attack, as it is common for the data entered by the user to appear verbatim on the results page. The next step is to test the field via the Burp Suite.



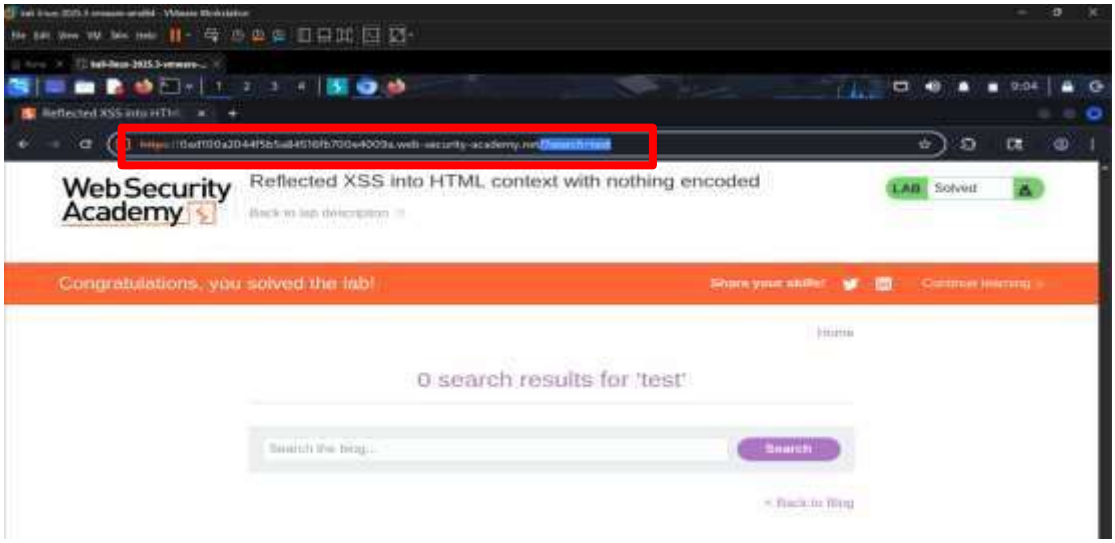
To analyse the traffic, we followed the same methodology as in the previous section (SQL Injection). Specifically, we enabled the

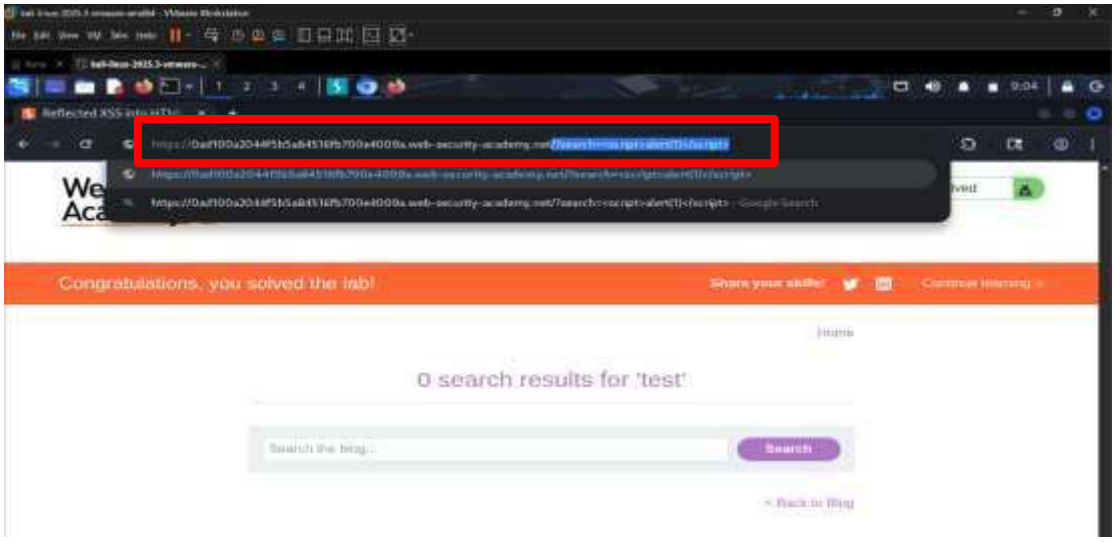
'**Intercept is** On' in Burp Suite and carried out a search within the application to generate traffic. The request was successfully intercepted, allowing us to examine and modify it before it reached the server.

Once we had identified the request carrying the search term, we sent it to the Repeater tool to carry out tests without affecting the browser. In Repeater, we replaced the original search term (**test**) with the classic XSS test payload '**<script>alert(1)</script>**'. The purpose of this script is to display a pop-up window (alert box) with the number '1', provided that the browser executes the code.

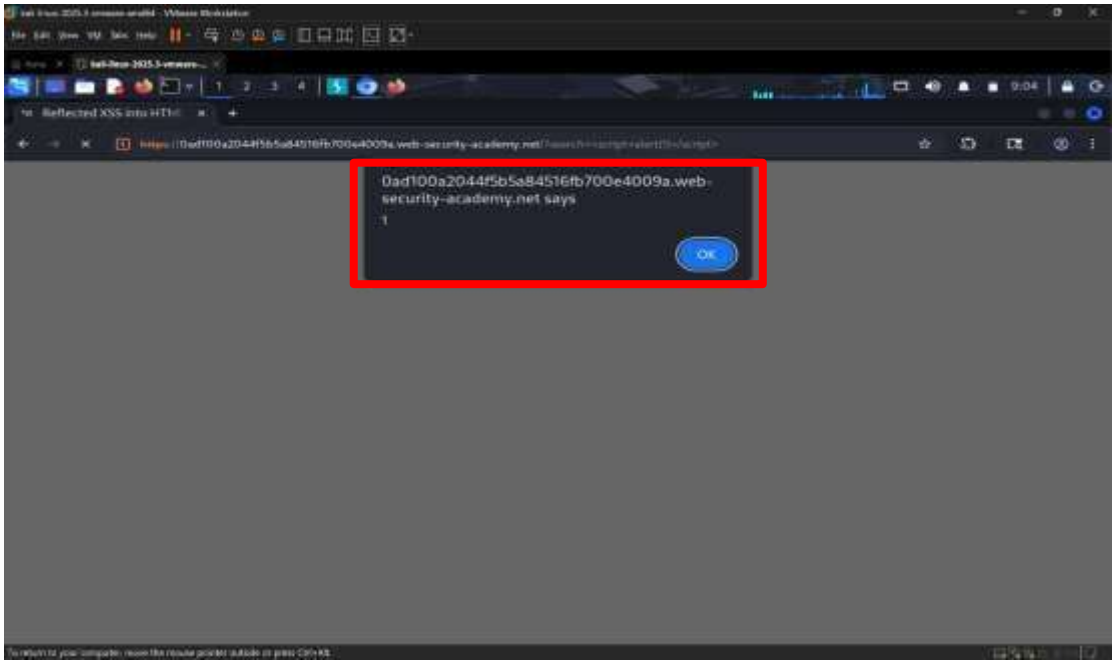


By clicking **'Send'**, we received the server's response. Examining the HTML code on the right-hand side (Response), we noticed that the application returned our script exactly

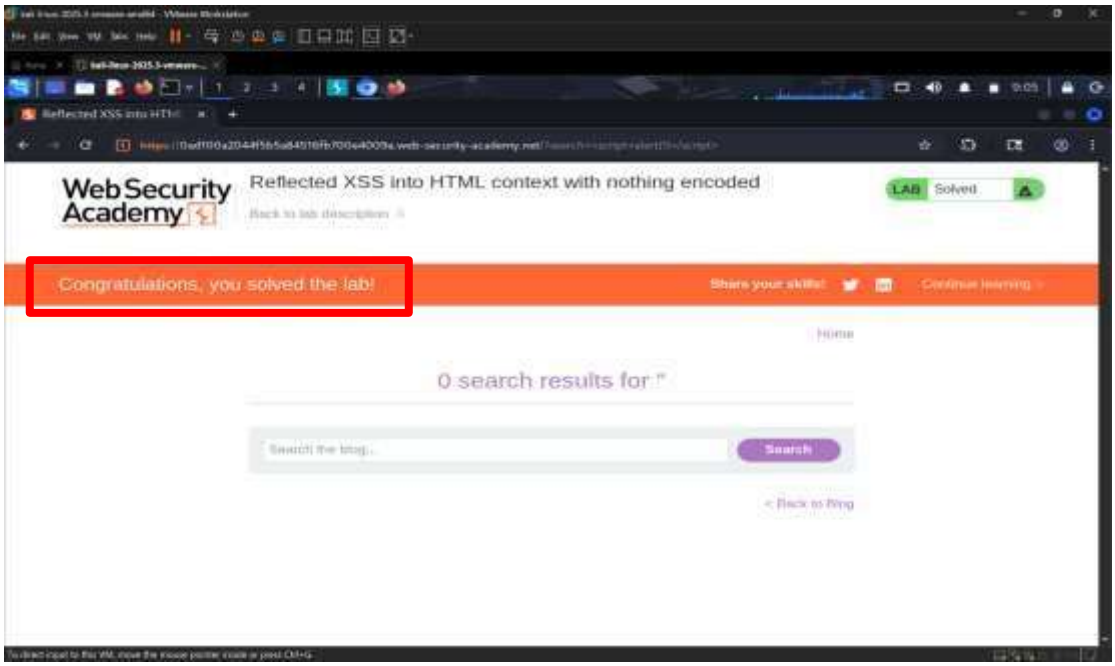




Having confirmed the vulnerability in the repeater, we proceeded to carry out the final attack in a real browser environment. First, we went to the 'Proxy' tab and disabled the **'Intercept is** Off' function to allow data to flow freely. We then opened the browser and manually modified the search parameter in the address bar, entering the payload **'/?search=<script>alert(1)</script>'**



Upon pressing Enter in the address bar, the browser sent the request and received the response containing our script. As shown in the image, the browser interpreted the <script> tags as a command to be executed rather than as plain text. The result was the display of a pop-up alert window with the value '1'. The appearance of this window is indisputable proof that the application is vulnerable to Cross-Site Scripting, as it allows JavaScript to be executed on the user's computer.



After closing the pop-up alert, the page loaded completely and the platform displayed the message ‘Congratulations, you solved the lab!’. This confirms, from the Web Security Academy’s perspective, that the attack was carried out correctly and the objective was achieved.

Conclusion on cross-site scripting attacks

In this scenario, we found that the application retrieves data from the user via the search bar and immediately returns it to the page’s code (HTML). This security vulnerability allows malicious users to inject arbitrary JavaScript code. Although in our example we used a harmless `alert()` command, in real-world scenarios an attacker could exploit the same vulnerability to steal cookies, leading to the compromise of user accounts.

References

- PortSwigger. (n.d.). Burp Suite Community Edition. https://portswigger.net/burp/communitydownload
- PortSwigger Web Security Academy. (n.d.). Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data. https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data
- PortSwigger Web Security Academy. (n.d.). Lab: Reflected XSS into HTML context with nothing encoded. https://portswigger.net/web-security/cross-site-scripting/reflected/lab-html-context-nothing-encoded
- PortSwigger. (n.d.). SQL injection. https://portswigger.net/web-security/sql-injection
- PortSwigger. (n.d.). Cross-site scripting (XSS). https://portswigger.net/web-security/cross-site-scripting
- Seven Seas Security. (5 May 2022). Cross-Site Scripting Lab Breakdown: Reflected XSS into HTML context with nothing encoded. https://www.youtube.com/watch?v=HfBiMStbWqY
- David Bombal Clips. (11 June 2024). Mastering Burp Suite: the ultimate web application hacking tool. https://www.youtube.com/watch?v=r46b9L6UQDo
- Netsec Explained. (20 September 2023). Master Burp Suite like a pro in just 1 hour. https://www.youtube.com/watch?v=QiNLNDSLuJY